

# MEI/MCA/MMC/MIA

# Shared memory programming (1)

## 1) Multithreading; 2) Thread scheduling;3) OpenMP basics

2026/27

*J.L. Sobral*

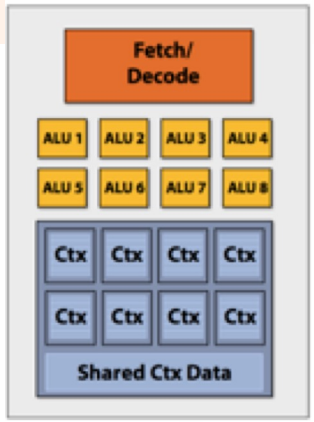
# Thread-level parallelism

**Levels of parallelism – software** vs **hardware**

Increasing parallelism granularity (#I per parallel activity)

- **Instruction-Level Parallelism (ILP)**
  - Parallel execution of multiple instructions
  - Vector processing (SIMD)
  - Automatic exploration by the hardware
  - Limited by program data and control dependencies

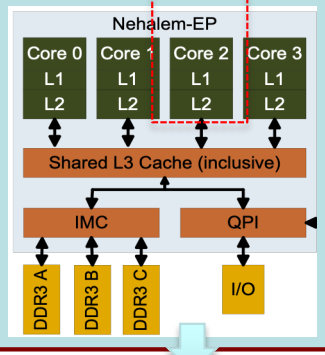
Instruction scheduling and mapping




~~C~~PU (single core)

- **Threads ["tasks"]**
  - Several threads executed in parallel
  - Limited by program dependencies (and by the algorithm)

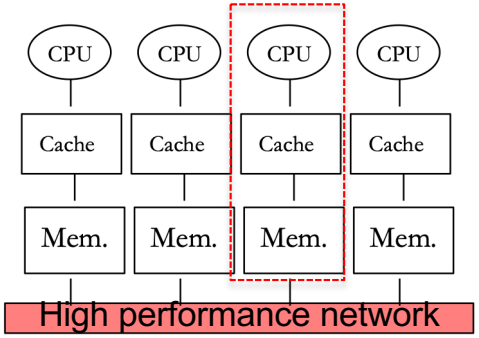
Thread scheduling and mapping




4 PUs (muti core)

Focus of today's lecture

- **Processes**
  - Multiple process from the same program or from several programs

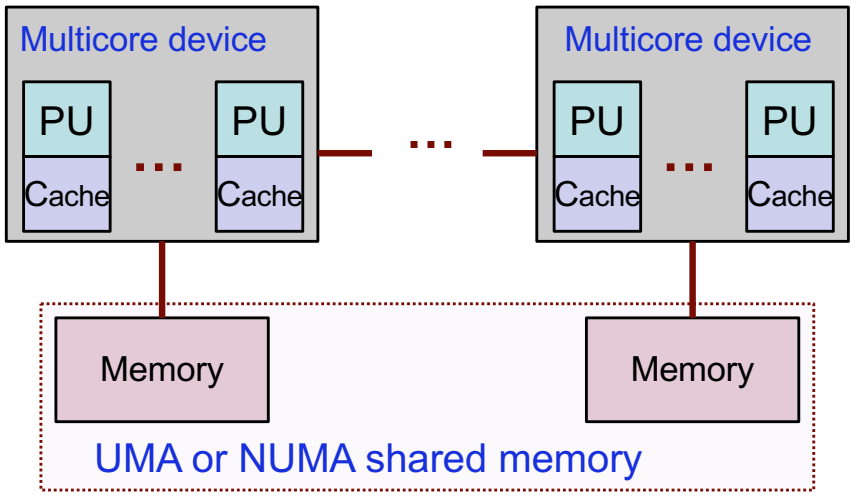



Cluster of multicore machines

# *Explicit parallel computing (1)*

## Shared memory parallel systems

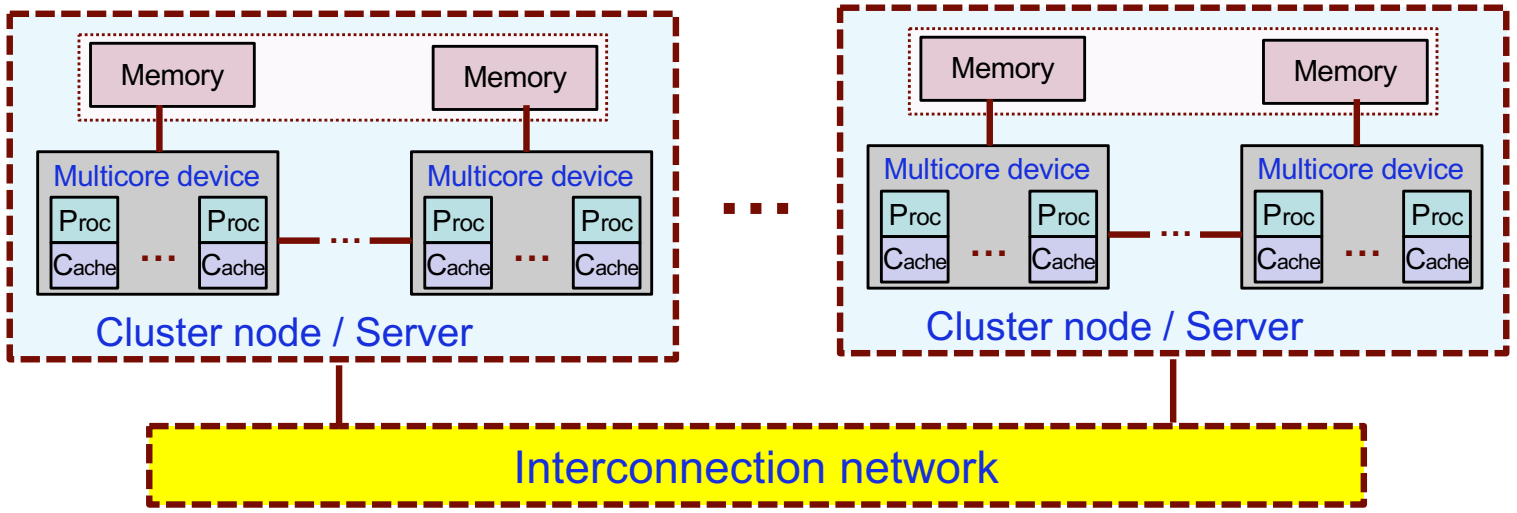
- parallelism on single or multiple devices *(same motherboard)*
  - **single physical memory address space**
  - each core can support multiple threads (SMT)
  - **memory bandwidth is shared by all cores**
  - **coherence of data in multiple caches?**



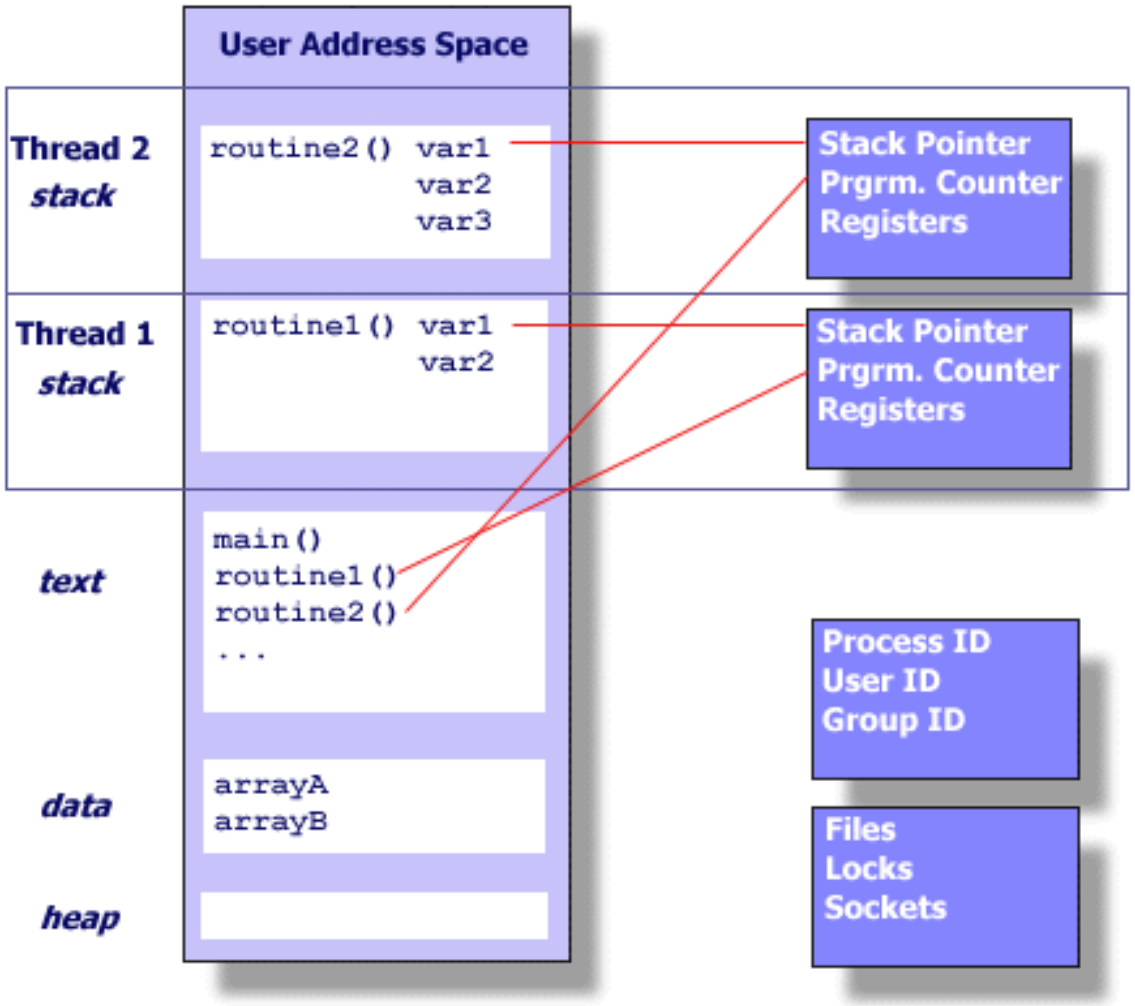
# *Explicit parallel computing (2)*

## Distributed memory parallel systems

- on multiple boards *(or multiple nodes/servers)*
  - each node with its private memory space
    - **memory bandwidth is proportional to the number of nodes**



# Programming abstractions example: a process with 2 threads

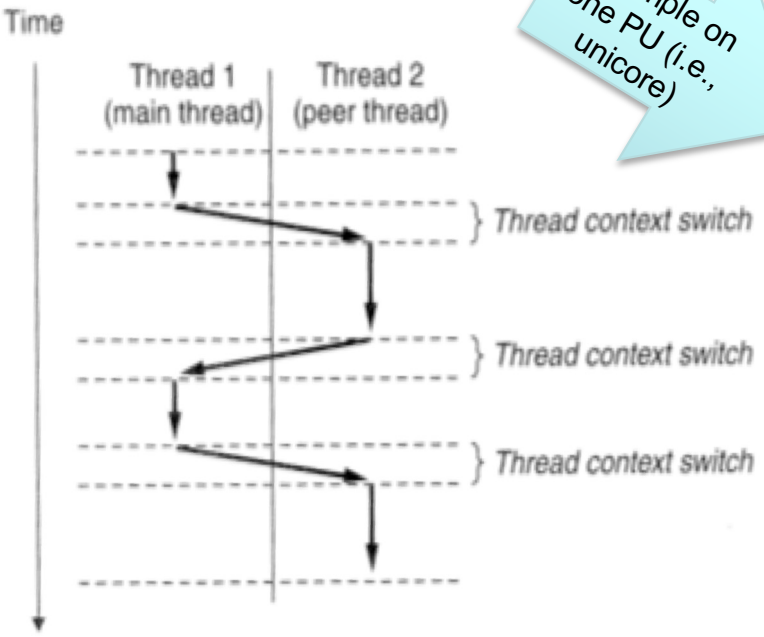


# Thread scheduling
## Logic vs physical parallelism

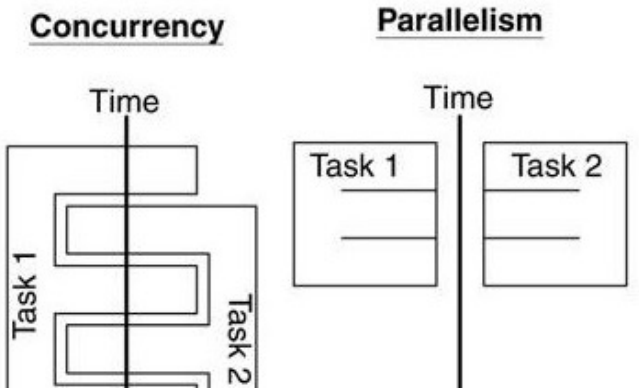
### Thread Scheduling (on 1 PU)

- The PU executes a set of threads, by scheduling blocks of instructions of each thread in time-slices (e.g. 50ms)
- It gives the illusion that threads are running in parallel

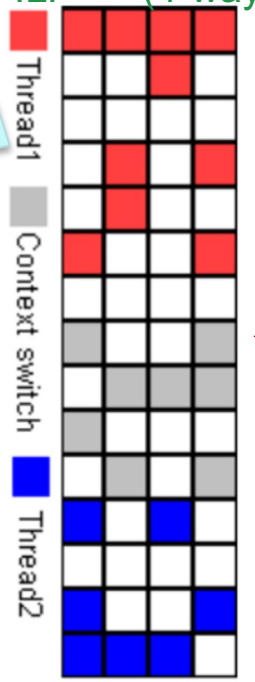
=> **Logic parallelism**

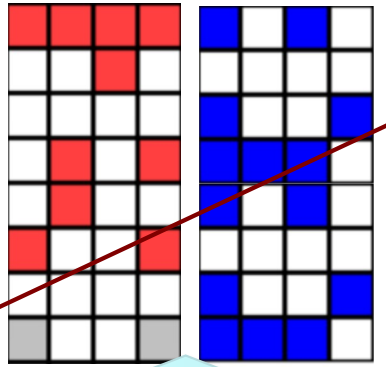



Example on one PU (i.e., unicore)




ILP => (4-way)



Example with 2 PUs

On current multicores, many threads are scheduled on multiple PUs

This scheduling and mapping has a **cost**:

- Thread creation
- Context switch
- Others (later…)

Scheduling result varies from execution to execution

### Physical parallelism

- At each clock cycle, instructions from multiple threads are running in parallel (on the same or in different PUs)

# Single-core Multithreading

## (Operating System vs hardware-based thread scheduling)

Executing multiple threads on a single core (PU)

- Share all resources but replicate registers, PC/IP, etc.
- Switching between threads might require saving current thread context and load new thread context (IP, etc…)

- **There are several ways to support multithreading**

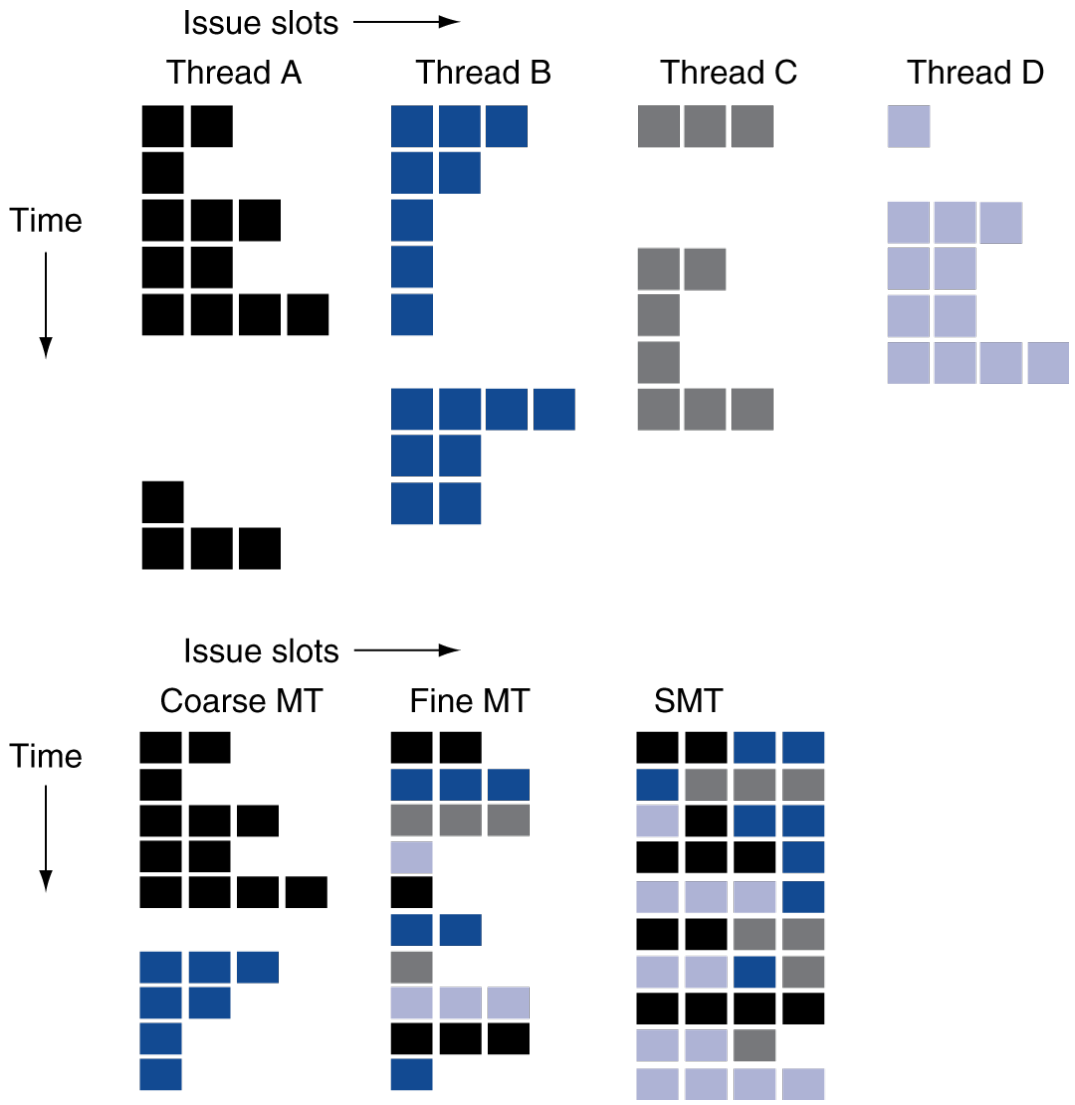
1. Coarse-grain multithreading [includes OS controlled]
   - Only switch on long stall (e.g., L2-cache miss) [or at the end of the time-slice]
   - Simplifies hardware, but doesn't hide short stalls
2. Fine-grain multithreading **=> on most GPU hardware**
   - Switch threads after each clock cycle
   - Interleave instruction execution
   - If one thread stalls, others are executed
3. Simultaneous multithreading *(SMT - next slides…)*

§ 7.5 Hardware Multithreading

# Multithreading Example

Example with 4-way ILP (support for execution of 4 instructions per clock cycle)

Alternative ways to support multithread execution (example with 4 threads)



# [NVIDIA] GPU Architecture

- Similarities to **vector machines**:
  - Works well with data-level parallelism
  - Supports scatter & gather memory transfers
  - Mask registers [*for control dependencies*]
  - **Large register files** → **Hardware-based thread scheduling**
    Registers are allocated to threads at creation
    - fast context switch (& lower cost)
    - constrains the maximum number of threads
    This is fine-grain multithreading
- Differences:
  - No scalar processor
  - **Uses multithreading to hide [memory] latency**
  - Has many functional units, as opposed to a few deeply pipelined units, behaving as a vector processor



Copyright © 2019, Elsevier Inc. All rights Reserved